

Project

Milan Lagae

University Name: Vrije Universiteit Brussel

Faculty: Sciences and Bioengineering Sciences

Course: GPU Computing

June 17, 2026

Contents

1. Intro	3
1.1. Declaration of AI Usage	3
2. Context	3
3. Pipeline & Genetic Algorithm	4
3.1. Overview	4
3.2. DEM	4
3.3. Preprocessing	4
4. Sequential	5
4.1. Updates	5
4.2. OpenMP	5
5. Parallel	6
6. Analysis	7
6.1. Methodology	7
7. Appendix	8
7.1. Platform	8
Bibliography	9

1. Intro

This report will discuss the assignment ‘Project’ for the course: ‘GPU Computing’. First, the context of the application of a Genetic Algorithm will be discussed for the problem space add hand. After the context, the pipeline for data preparation and more information about the problem will be discussed in Section 3. Continuing from the pipeline, the updated sequential will be discussed including the naive speedup methods. Finally, the focus of the assignment the parallel version in Section 5. Concluding, both version will be analyzed in Section 6.

1.1. Declaration of AI Usage

For starting the project and the start up phases AI was used. The repository for preprocessing the DEM data can be found here [1]. As a starting point, the following repository was used for both the sequential & parallel version of the algorithm: [2]. The initial repository was updated to reflect the modern features, such as the deprecated `rand()` function and more. After the code was updated, some AI help was used to think through the genetic algorithm structures.

2. Context

The problem for which the genetic algorithm is used, is a problem widely occurring in the wireless networking world (**TODO: source**). Where, wireless internet providers must decide where to position there service towers to provided the maximum or greatest coverage, taking into account terrain features.

This identical problem, occurs in the military domain, were with a limited set of sensors, the greatest coverage (or detection probability) must be gained with the available sensors (**TODO: source**). This type problem is considered an **TODO: add**, where the problem is to large to compute with normal methods and therefor different methods are found to look for optimal algorithms: (**TODO: source**). For this reasons, there have been historically been several attempts with success add using Genetic Algorithms to solve this particular problem (**TODO: source**).

There is also a variation on this problem, where two types of sensors are placed on the ground in hilly terrain and are used to check detection of vehicles trying to pass (**TODO: source**). The algorithm is used to place two types of sensors: FLIR & Seismic.

For this reason, this paper is the implementation of a scoped down version of the genetic algorithm in(**TODO: source**). It uses only one type of sensor FLIR placed, 1.8m above the ground, illustration can be seen in (**TODO: add image**).

3. Pipeline & Genetic Algorithm

This section will discuss the pipeline of preparing and retrieving required DEM (Digital Elevation Model) data and the outputs the python preprocessing scripts generated for use in the genetic algorithm.

3.1. Overview

TODO: Add overview image from slides

3.2. DEM

The genetic algorithm itself requires appropriate terrain data for a more realistic scenario. This type of data can be downloaded from public sources easily, and for this reason, the data for the country of Belgium was used. In this case, the source of the data came from (**TODO: source**) and was downloaded on (**TODO: add**).

3.3. Preprocessing

After the DEM data is available, the preprocessing step using `python` may begin. For this type of data & problem, there is a library available which does the heavy preprocessing lifting that is required, called: `gdal` [3].

The repository which includes the preprocessing files can be found at [1]. A short description and workings of the preprocessing will be made here. The repository consists of the following files: `preprocess_dem.py` , `preprocess_viewshed.py` , `conver_sensors.py` , `check_sizes.py` , `check_visibility.py` .

The `preprocess_dem.py` is responsible for converting a given square area based on ROI (Gdal coordinates) into a binary file containing area elevation data. The script creates several output files, some are important for important into QGIS but the two important files for the GA are: `elevation.bin` and `elevation_meta.txt` .

The viewshed is generated by `preprocess_viewshed.py` file – it uses the `gdal.ViewshedGenerate` function to calculate the visibility of a source sensor 1.8m heigh and 0m (on the ground). Depending on the problem, these parameters can be tuned. This scripts also exports several files, two files matter for the GA: `viewshed_lut.bin` and `viewshed_meta.txt` .

There are two verification files included: `check_sizes.py` this one checks the size of each cell depending on the size of the area & the number of cells of the grid; `check_visibility.py` uses Gdal functions to check the general visibility of the area by reading in the `viewshed_lut.bin` file.

Closing the list of scripts: `convert_sensors.py` is responsible for converting the `*-sensors.csv` file generated by the GA algorithm from the coordinates on the GRID to GEO based coordinates for import and visualization in QGIS.

4. Sequential

This section will briefly discuss the updates made to the sequential genetic algorithm and naive attempt at speedup up the code by using OpenMP¹.

4.1. Updates

4.2. OpenMP

¹<https://www.openmp.org/>^o

5. Parallel

6. Analysis

6.1. Methodology

7. Appendix

7.1. Platform

The benchmarks were executed on a KUbuntu 25.04 desktop, with the specifications listed in Table 1.

PART	VALUE
CPU	Ryzen 9 5950X
GPU	RTX 3070
Driver Version	595.97
RAM	64GB (3200 Mhz)
OS	Windows 11 - Version 10.0.22631 Build 22631
WSL Version	2
WSL Distro	24.04
Kernel Version	6.6.87.2-microsoft-standard-WSL2
NVCC	13.2, V13.2.51
NCU	Version 2026.1.0.0 (build 37166530)
GCC	TODO
OpenMP	TODO

Table 1: Desktop Specifications

Bibliography

- [1] M. Lagae, “DEM-Data-Preprocessing.” [Online]. Available: <https://github.com/gaMingLT/DEM-Data-Preprocessing>^o
- [2] B. Raymond Chee, “Final Project for 15-618 Parallel Computing.” [Online]. Available: https://github.com/rjchee/parallel_genetic_algorithms^o
- [3] Frank Warmerdam Even Rouault and others, “GDAL.” [Online]. Available: <https://gdal.org/en/stable/>^o